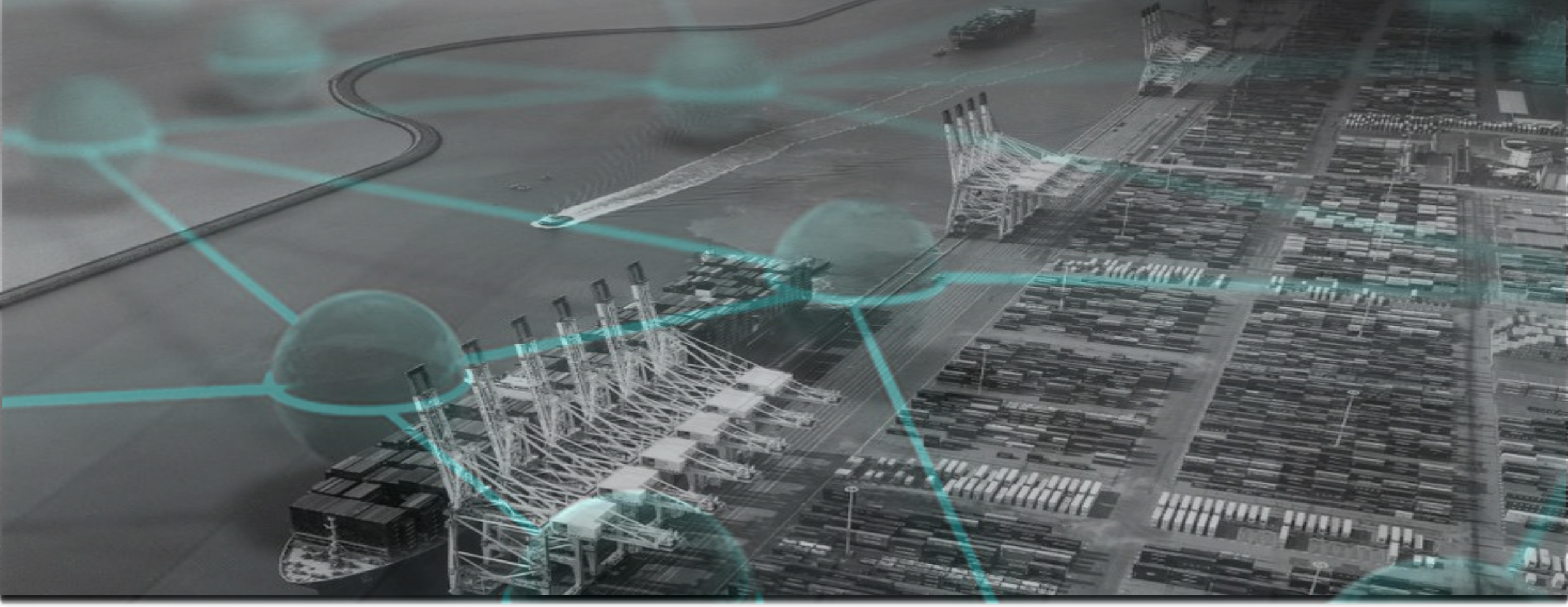


BLOCKCHAIN

PROGRAMMATION DE SMART CONTRACTS AVEC ETHEREUM ET HARDHAT

Claude Duvallet

Université Le Havre Normandie – UFR ST

Université de Sfax

PLAN

1. Présentation et installation des outils
   1. Node JS
   2. Hard Hat
   3. Metamask
   4. Visual Code
2. Manipulations des smart contacts

1 — PRÉSENTATION ET INSTALLATION DES OUTILS

NODE JS

HARDHAT

HARDHAT

- Un outil de développement particulièrement apprécié dans le monde des blockchains qui vise à remplacer Ganache et Truffle,
- Plus spécifiquement pour la création et les tests de contrats intelligents (smart contracts) sur la plateforme Ethereum.
- Fonctionnalités :
  - Créer de nouveaux contrats intelligents en Solidity.
  - Compiler ces contrats pour les rendre exécutables sur une blockchain.
  - Tester de manière exhaustive le fonctionnement de ces contrats dans un environnement simulé.
  - Déployer les contrats sur un réseau de test ou principal.
  - Déboguer les éventuels problèmes rencontrés.

POURQUOI HARDHAT

- **Simplicité d'utilisation :**
  - une interface de ligne de commande intuitive et une configuration minimale,
  - accessible aux débutants comme aux développeurs expérimentés.
- **Flexibilité :**
  - Hautement personnalisable grâce à un système de plugins
  - Pour étendre ses fonctionnalités en fonction de vos besoins spécifiques.
- **Performance :**
  - Optimisé pour offrir des temps de compilation et d'exécution rapides,
  - Ce qui accélère considérablement votre cycle de développement.
- **Communauté active :**
  - Une communauté de développeurs très active,
  - Une documentation abondante et une assistance rapide en cas de besoin.

HARDHAT : PRINCIPALES FONCTIONALITÉS

- **Environnement de développement local :**
  - HardHat crée un réseau Ethereum local pour tester vos contrats dans un environnement isolé.
- **Gestion des tâches :**
  - Automatisation des tâches répétitives comme la compilation, les tests et le déploiement.
- **Intégration avec des outils populaires :**
  - Il s'intègre facilement avec d'autres outils comme Truffle, Ethers.js et Waffle.
- **Support de Solidity :**
  - Un support complet pour le langage de programmation Solidity, utilisé pour écrire des contrats intelligents.

HARDHAT : INSTALLATION

1. Ouvrez un terminal et placez-vous dans un répertoire de travail.
2. Créer un nouveau répertoire :

```
mkdir my-hardhat-project
```

3. Accéder au répertoire :

```
cd my-hardhat-project
```

HARDHAT : INSTALLATION

4. Initialiser un nouveau projet au moyen de la commande suivante qui va créer un fichier « package.json ».

```
npm init -y
```

```
{
  "name": "my-hardhat-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

HARDHAT : INSTALLATION

5. Installez maintenant le package HardHat au moyen de la commande suivante :

   `npm install --save-dev hardhat`

6. Vous pouvez ensuite initialiser votre application :

   `npx hardhat init`

```
888     888                          888 888                888
888     888                          888 888                888
888     888                          888 888                888
8888888888  8888b.  888d888 .d88888 88888b.   8888b.      888888
888     888     "88b 888P"  d88" 888 888 "88b     "88b 888
888     888 .d888888 888    888  888 888  888 .d888888 888
888     888 888  888 888    Y88b 888 888  888 888  888 Y88b.
888     888 "Y888888 888     "Y88888 888  888 "Y888888  "Y888

Welcome to Hardhat v2.22.2

? What do you want to do? …
▸ Create a JavaScript project
  Create a TypeScript project
  Create a TypeScript project (with Viem)
  Create an empty hardhat.config.js
  Quit
```

HARDHAT : INSTALLATION

- Si le menu s'affiche, sélectionnez l'option JavaScript.
- Lorsque la phrase suivante apparaît "Do you want to install this sample project's dependencies with npm (@nomicfoundation/hardhat-toolbox)?", tapez Y.
- La Hardhat Toolbox permet d'ajouter un nombre conséquent d'utilitaires à l'application tels que Ethers, Mocha, Chai…

HARDHAT : DÉMARRAGE

- Utilisez la commande suivante :

```
npx hardhat node
```

- Résultat :

```
Started HTTP and WebSocket JSON-RPC server at
http://127.0.0.1:8545/

Accounts

========

WARNING: These accounts, and their private keys, are
publicly known.

Any funds sent to them on Mainnet or any other live network
WILL BE LOST.


Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
(10000 ETH)

Private Key:
0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4
f2ff80


Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
(10000 ETH)

Private Key:
0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b
78690d
```

HARDHAT : STRUCTURE D'UN PROJET

- La structure de base d'un projet HardHat est la suivante :
  - contracts/ : ce répertoire contient vos contrats en langage Solidity.
  - test/ : ce répertoire contient vos tests unitaires.
  - hardhat.config.js : il s'agit du fichier de configuration principal.
- Contenu du fichier hardhat.config.js

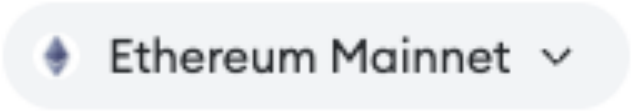
```
require("@nomicfoundation/hardhat-toolbox");

/** @type import('hardhat/config').HardhatUserConfig */
module.exports = {
  solidity: "0.8.24",
};
```

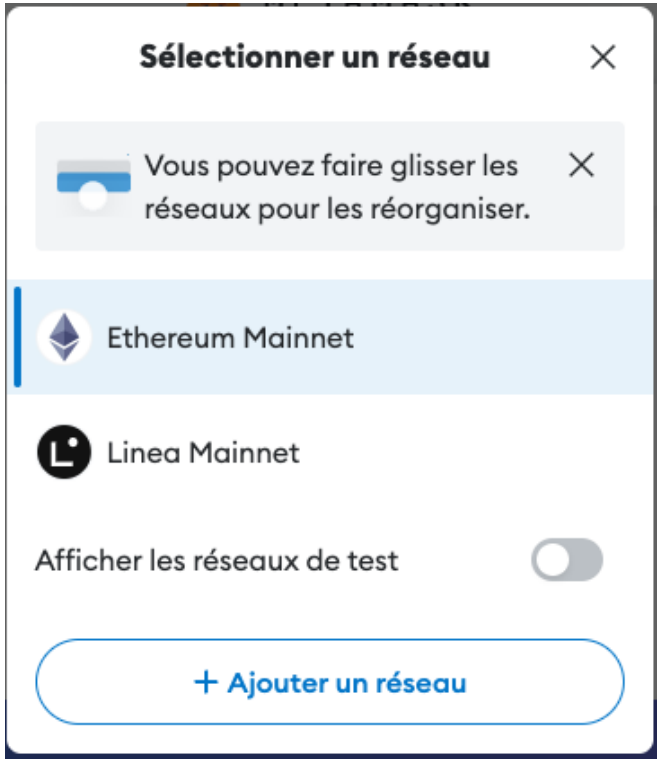
METAMASK

METAMASK : CONFIGURATION

- Sélection d'un réseau Blockchain
  - Par défaut, c'est le réseau principal qui est sélectionné mais nous ne travaillerons pas sur ce réseau.

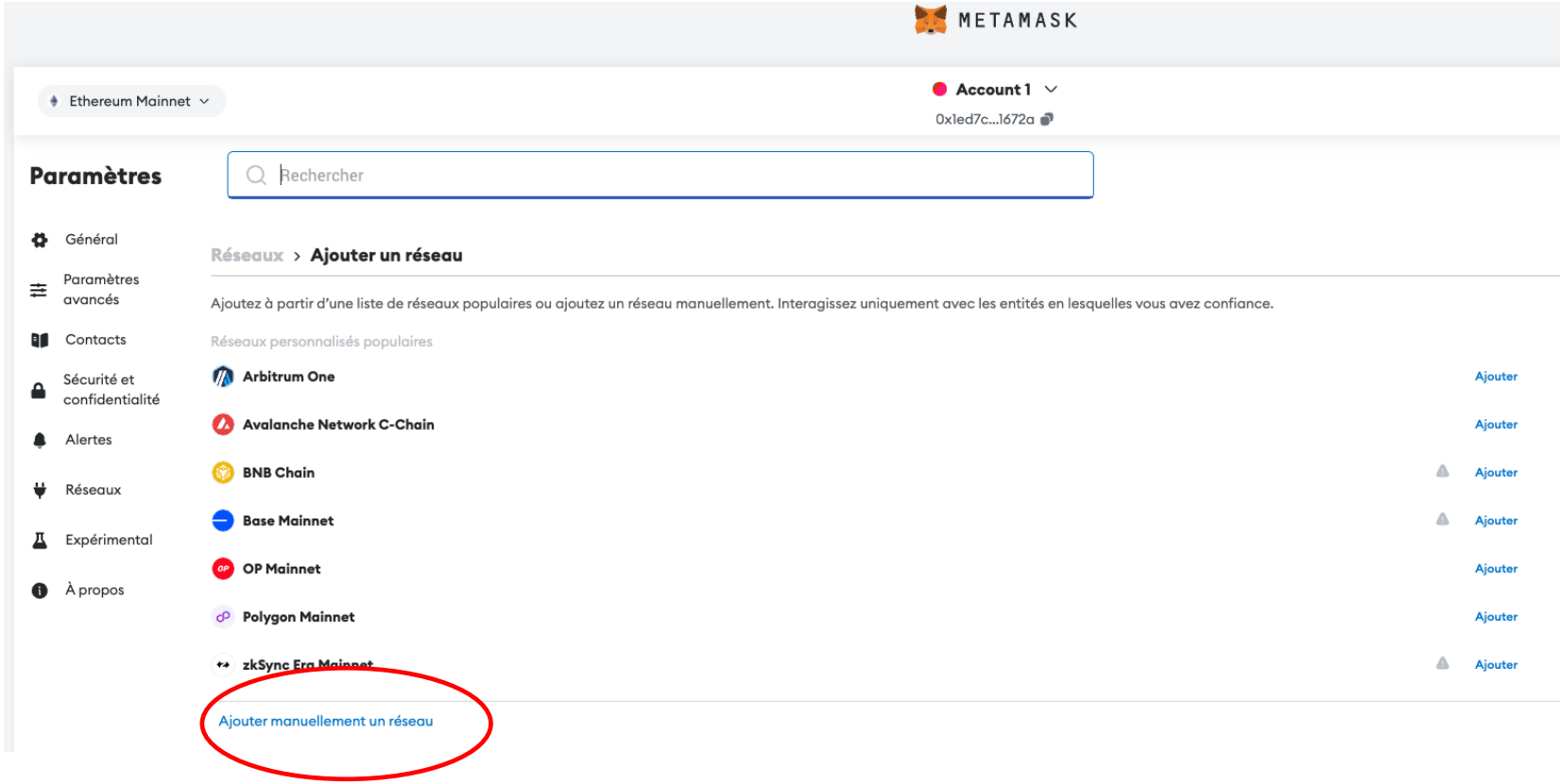


  - Nous allons travailler sur un réseau local (celui de HardHat) que nous allons configurer manuellement (ajouter un réseau).



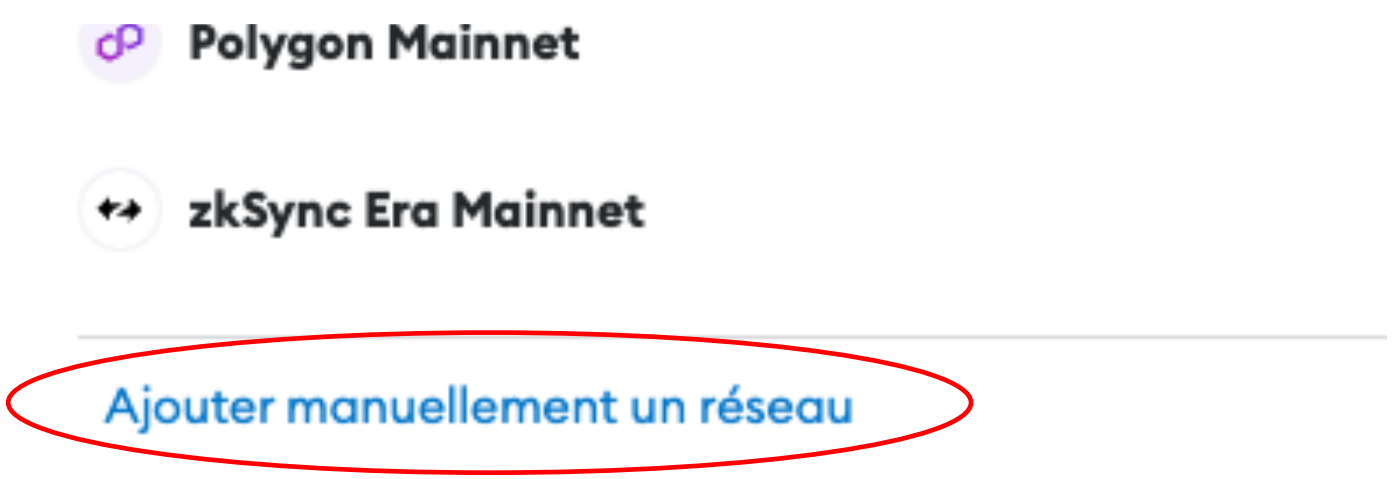
METAMASK : CONFIGURATION

- Ajout d'un réseau Blockchain



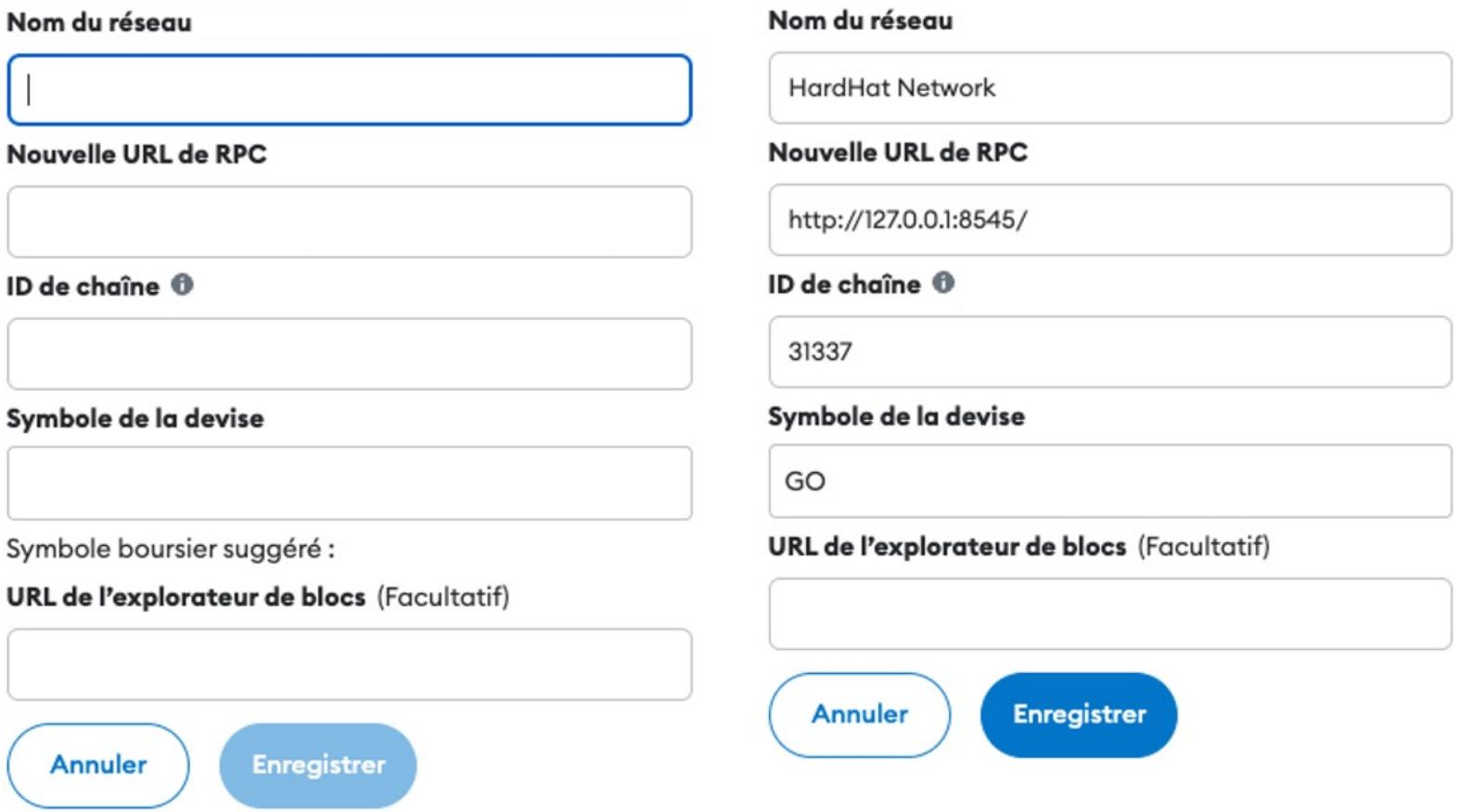
METAMASK : CONFIGURATION

- Ajout d'un réseau Blockchain



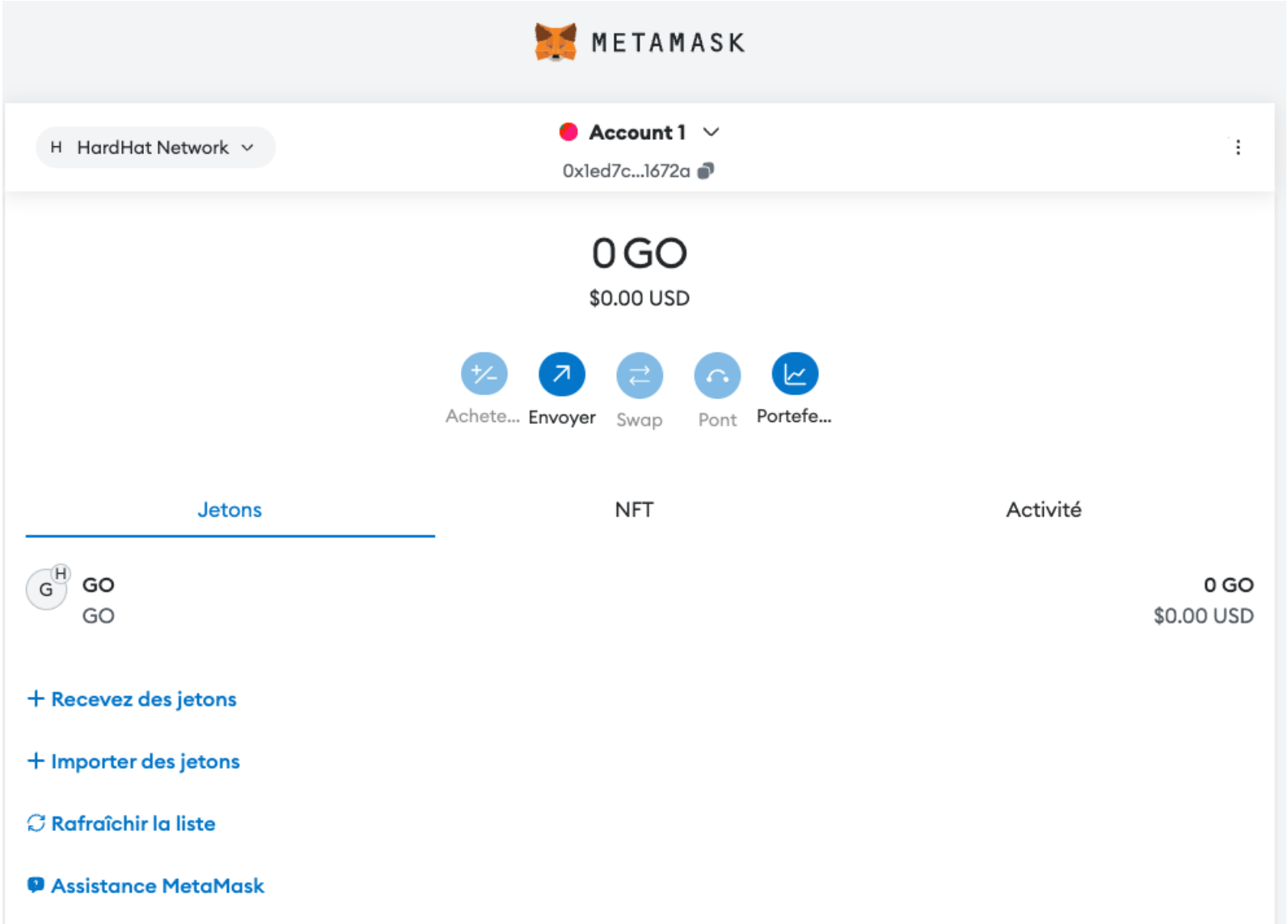
METAMASK : CONFIGURATION

- Ajout d'un réseau Blockchain



METASMASK

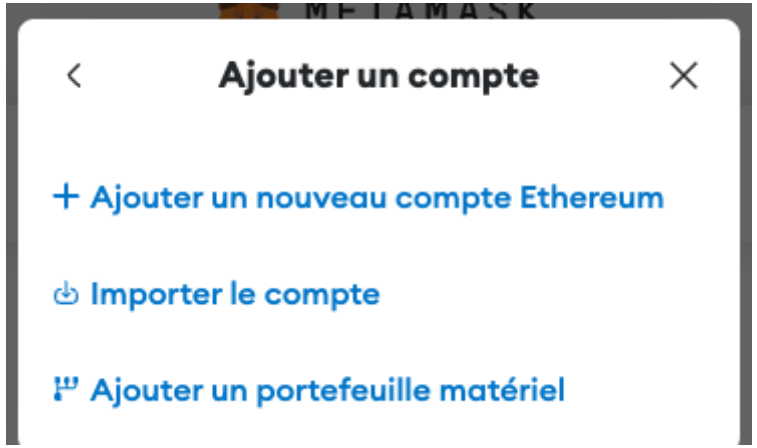
- Importation de comptes HardHat



METAMASK : CONFIGURATION

- Au niveau de « Account 1 », on va
  - sélectionner « Ajouter un compte ou un portefeuille matériel »
  - puis choisir « Importer le compte »
  - et configurer ce compte avec l'une des clefs privées de compte du réseau local « HardHat »





METAMASK : CONFIGURATION

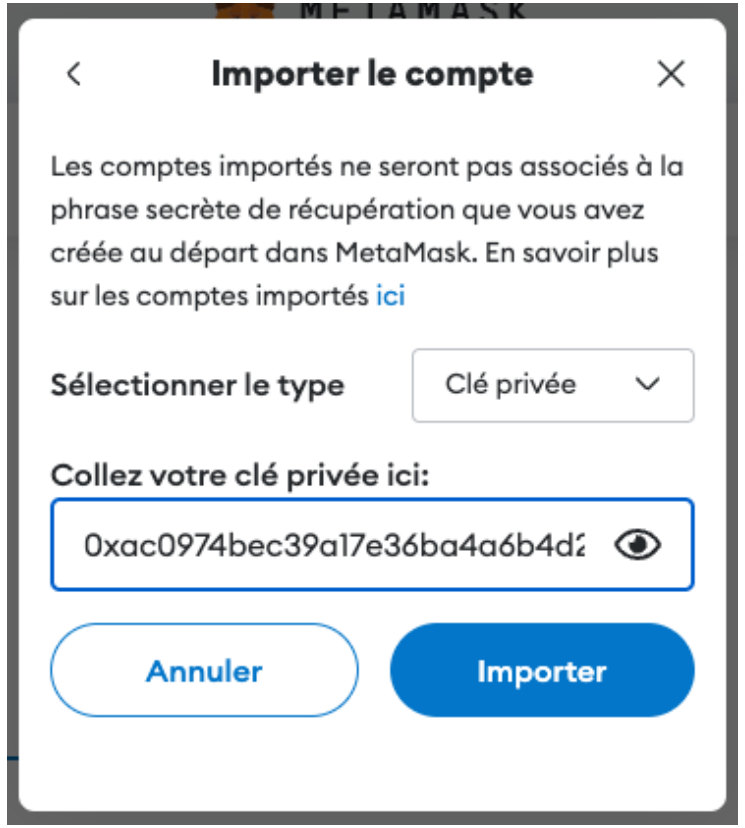
- Configurer ce compte avec l'une des clefs privées de compte du réseau local « HardHat »

```
Accounts
========

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

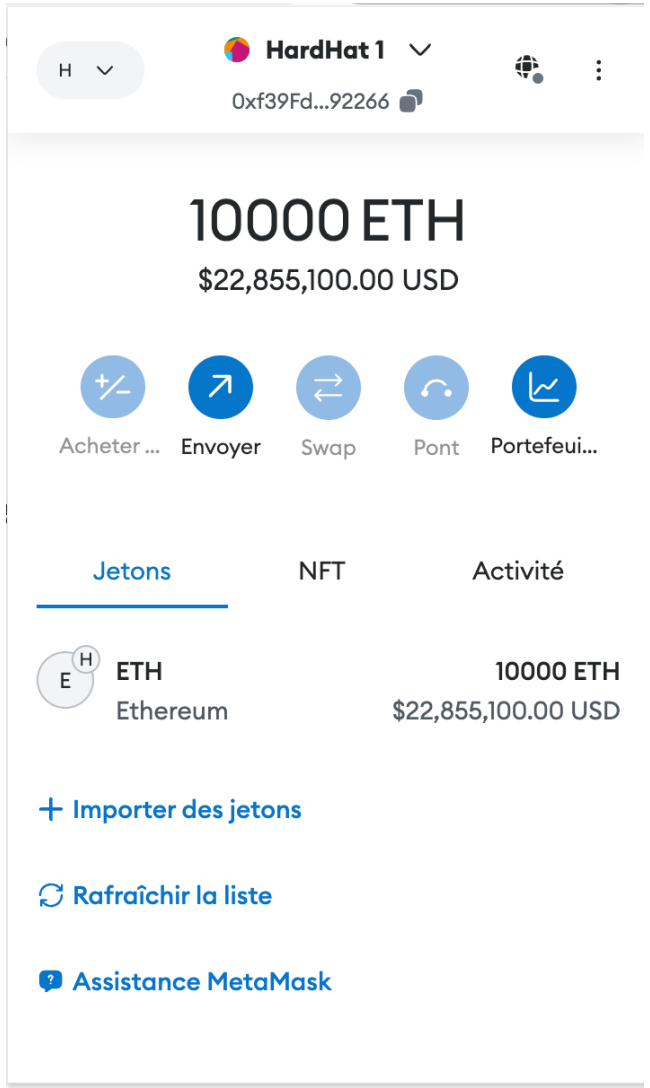
Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
```



METAMASK : CONFIGURATION

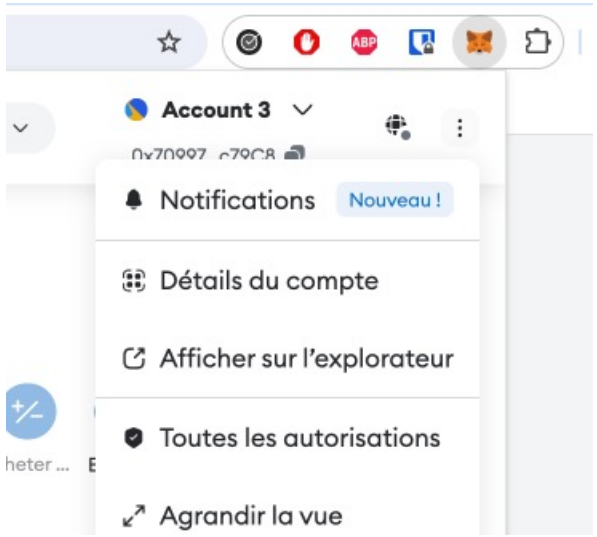
- Dans Metamask, nous avons désormais des comptes avec 10000 ETH



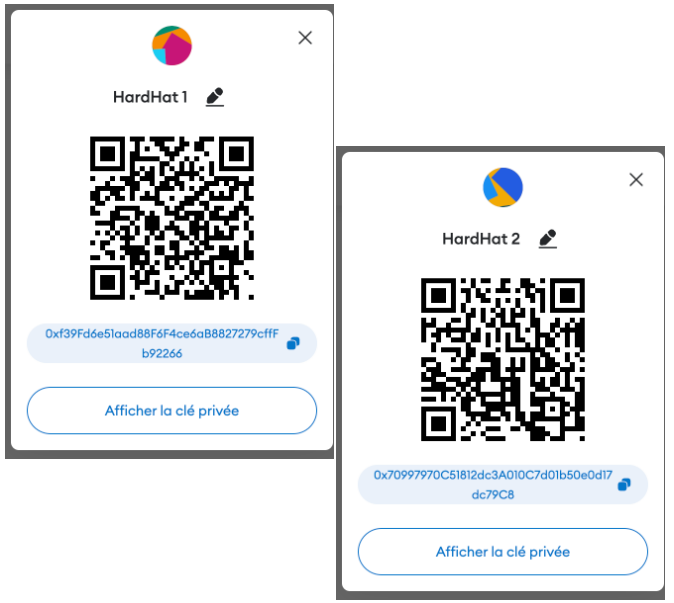
METAMASK : ACTIVATION DE L'EXTENSION

- Vous pouvez activer l'extension MetaMask pour qu'elle soit visible sur le navigateur,





- Et choisir de modifier les détails pour redéfinir le nom des comptes.



VS CODE

2 – MANIPULATION DES SMART CONTRACTS

CRÉATION D'UN PREMIER CONTRAT INTELLIGENT (SMART CONTRACT)

- Un Smart Contract écrit en Solidity possède toujours une en-tête spécifiant la version du compilateur Solidity choisie.
- Dans notre cas, nous allons choisir une version du compilateur comprise entre la 0.4.16 inclue et la 0.9.0 exclue.
- Il ne faut pas oublier d'ajouter la licence sinon le Smart Contract ne compilera pas.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.16 <0.9.0;

contract IncredibleStorage {}
```

CODE DU SMART CONTRACT

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.16 <0.9.0;

contract IncredibleStorage {
    uint awesomeUInt;

    function set(uint x) public {
        awesomeUInt = x;
    }

    function get() public view returns (uint) {
        return awesomeUInt;
    }
}
```

CODE DU SMART CONTRACT (SUITE)

- Il faut sauvegarder ce code dans le dossier « `contracts` » sour le nom « `IncredibleStorage.sol` »
- Nous pouvons aussi ajouter une valeur par défaut lors du déploiement. Il faut alors ajouter un constructeur :

```
constructor(uint _awesomeUInt) {
    awesomeUInt = _awesomeUInt;
}
```

DÉPLOIEMENT DU SMART CONTRACT

- Pour pouvoir déployer un Smart Contract avec Hardhat, nous devons créer un script dans le dossier **`ignition/modules`** que nous allons appeler **`IncredibleStorage.js`**.

```
touch ./ignition/modules/IncredibleStorage.js
```

- Le code du script de déploiement de notre Smart Contract :

```
const { buildModule } = require("@nomicfoundation/hardhat-ignition/modules");

module.exports = buildModule("IncredibleStorageModule", (m) => {
  const incredibleStorage = m.contract("IncredibleStorage", [42], {  });

  return { incredibleStorage };
});
```

DÉPLOIEMENT DU SMART CONTRACT

- `m.contract` prend en paramètre le nom du Smart Contract, les variables à passer au constructeur et finalement les variables correspond aux paramètres du Smart Contract lors du déploiement.
- Attention lorsque vous utilisez la commande ignition, il est important de préciser le réseau utilisé car sinon vous n'allez pas pointé sur le bon.
- Pour déployer le contrat, on utilise alors la commande suivante :

```
npx hardhat ignition deploy
./ignition/modules/IncredibleStorage.js --network
localhost
```

DÉPLOIEMENT DU SMART CONTRACT : RÉSULTAT

```
Compiled 2 Solidity files successfully (evm target: paris).
Hardhat Ignition 🚀

Deploying [ IncredibleStorageModule ]

Batch #1
  Executed IncredibleStorageModule#IncredibleStorage
[ IncredibleStorageModule ] successfully deployed 🚀

Deployed Addresses


IncredibleStorageModule#IncredibleStorage -
0x5FbDB2315678afecb367f032d93F642f64180aa3
```

DÉPLOIEMENT DU SMART CONTRACT : RÉSULTAT

```
eth_call
  Contract deployment: <UnrecognizedContract>
  Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266

eth_sendTransaction
  Contract deployment: <UnrecognizedContract>
  Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
  Transaction:
0xc3ed171d65396b80b2b240f9dcec98b2fa9765022a5f2be7676d79a33bd6f015
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  Value:               0 ETH
  Gas used:            150410 of 150410
  Block #1:
0xab8539c65dd3b2015038064a7d65f3a057a67e83a1046fd467642712a28ea8a6
```

- Il est possible d'avoir **UnrecognizedContract** à la place de **IncredibleStorage** mais rien de grave.
- Le Smart Contract **IncredibleStorage** a été déployé.

TESTER LE SMART CONTRACT DEPLOYÉ

- Pour tester notre contrat, nous devons ajouter un fichier de tests :

`./test/IncredibleStorage.js`

- Dans ce fichier, ajoutez le code du fichier `IncredibleStorageTest.js`
- Ce code nous servira de base pour écrire nos tests.
- Ici, nous avons utiliser la bibliothèque `chai` qui nous permet d'avoir des helpers pour construire nos tests unitaires.

TESTER LE SMART CONTRACT DEPLOYÉ

- Ajoutons un test pour vérifier que notre valeur par défaut (transmise lors de la création du Smart Contract) est bien attribuée :
- Fichier : test/IncredibleStorage.js
- On exécute les tests :

```
$ npx hardhat test

IncredibleStorage
    ✓ Should have a default value (357ms)
```

INTÉGRATION DANS UNE APPLICATION REACT

- Création d'un nouveau projet React :

```
npx create-react-app incredible-storage-dapp
cd incredible-storage-dapp
```

- Installation des dépendances nécessaires :

```
npm install ethers react
```

- Créer une application React similaire à celle que l'on a produit pour le Smart Contract déployé avec Ganache et Truffle

MANIPULATION DE SMART CONTRACT AVEC FOUNDRY ET VYPER

FOUNDRY ET VYPER

- Foundry est un ensemble d'outils permettant de développer des Smart Contracts.
  - C'est une alternative à Hardhat.
- Vyper est une alternative à Solidity, un Snake Contract !
- Pour installer Foundry :

```
curl -L https://foundry.paradigm.xyz | bash
foundryup
```

FOUNDRY ET VYPER

- Pour installer Vyper :

```
pip install vyper
```

- Vérifiez que Vyper est bien installé :

```
vyper –version
0.4.0+commit.e9db8d9
```

CRÉATION DU SMART CONTRACT

- Utiliser la commande suivante pour créer un projet Foundry :

```
forge init storage_snake
cd storage_snake
```

- Les contrats se situent dans le répertoire **src**
- Les scripts de déploiement dans script
- Et les tests dans test.
- Créez un fichier en le nommant SnakeStorage.vy :

```
touch ./src/SnakeStorage.vy
```

```
storage_snake/
├── .github/
│   └── workflows/
│       └── test.yml
├── src/
│   └── Counter.sol
├── test/
│   └── Counter.t.sol
├── script/
│   └── Counter.s.sol
├── .gitignore
├── foundry.toml
└── README.md
```

LE FICHIER SNAKESTORAGE.VY

```
#pragma version >=0.3.10

awesomeUInt: uint256

@external
def set(x: uint256):
    self.awesomeUInt = x

@external
def get() -> uint256:
    return self.awesomeUInt
```

COMPILATION DU SMART CONTRACT

- Maintenant, nous allons le compiler :

```
forge build
```

- Vous devriez avoir le résultat suivant :

```
[⠒] Compiling...
[⠢] Compiling 1 files with Vyper 0.4.0
[⠆] Compiling 27 files with Solc 0.8.26
[⠆] Vyper 0.4.0 finished in 149.99ms
[⠄] Solc 0.8.26 finished in 2.12s
Compiler run successful!
```

DÉPLOIEMENT DU SMART CONTRACT

- Voici le script de déploiement :

```
import {Script} from "forge-std/Script.sol";

contract CounterScript is Script {
    function run() public {
        vm.broadcast();
        deployCode("src/SnakeStorage.vy");
    }
}
```

DÉPLOIEMENT DU SMART CONTRACT

- Avant de déployer le Smart Contract, vous devez lancer un nœud Ethereum.
- Pour se faire, Foundry fournit Anvil.
- Dans un terminal, exécutez cette commande : **`anvil`**.
- Vous pouvez maintenant déployer le Smart Contract :

```
forge create --rpc-url http://127.0.0.1:8545 --private-key
0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
src/SnakeStorage.vy:SnakeStorage
```

- Les arguments utilisés sont :
  - **`rpc-url`** : l'adresse RPC de votre noeud Ethereum.
  - `private-key` : la clé privée à utiliser pour déployer le Smart Contract. Elles sont données à chaque démarrage d'Anvil.

DÉPLOIEMENT DU SMART CONTRACT

- Si tout va bien, Anvil a dû afficher quelque chose de semblable :

```
eth_getTransactionReceipt
  Transaction: 0x3e20f4a9fc55a3bdbf86b3e0595083f16263ba4be8dab4b6c7d94e83c95f4908
  Contract created: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  Gas used: 75340

  Block Number: 1
  Block Hash: 0xf6055d2bf4c5cb543195cde777bc34827bc21c0f252ec4b7699920ffc7f4139e

  Block Time: "Thu, 22 Aug 2024 13:35:06 +0000"
```

INTÉGRATION DANS UNE APPLICATION REACT

- Création d'un nouveau projet React :

```
npx create-react-app storage-snake-dapp
```

```
cd storage-snake-dapp
```

- Installation des dépendances nécessaires :

```
npm install ethers react
```

- Créer une application React similaire à celle que l'on a produit pour le Smart Contract déployé avec Ganache et Truffle

TRAVAUX PRATIQUES

- Créer un smart contract permettant
  - d'enregistrer une ou plusieurs chaînes de caractères dans la blockchain
  - de récupérer la liste des chaînes présentes dans le Smart Contract
- Construisez l'application Web qui permet d'interagir avec ce Smart Contract
- Etendre cette application pour hacher un fichier quelconque et enregistrer ce hash dans la Blockchain
- Permettre à un utilisateur de vérifier si le hash d'un fichier est déjà présent dans la Blockchain


MERCI POUR VOTRE ATTENTION